



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 07

No de Prácticas(s): 5

Integrante(s): 323078401

323115296

323082392

323139441

323253916

No. de lista o

brigada: 04

Semestre: 2027-I

Fecha de entrega: 02/10/2026

Observaciones: _____

CALIFICACIÓN: _____

Índice

1. Introducción	2
1.1. Planteamiento del problema	2
1.2. Motivación	2
1.3. Objetivos	2
2. Marco Teórico	2
3. Desarrollo	3
3.1. Actividad 1: Análisis de las respuestas de las LLMs	3
3.2. Actividad 2: Programa Empleado	5
3.3. Actividad 3: Programa Fecha	6
4. Resultados	7
4.1. Resultados de la Práctica 5	7
5. Conclusiones	10
6. Referencias	10

1. Introducción

1.1 Planteamiento del problema

En esta práctica se requiere implementar la solución a dos problemas: El primero pide crear un constructor para el concepto de empleado con sus atributos, nombre, apellido y salario mensual, además de instanciar dos objetos de esa clase para asignar y modificar sus salarios mensuales y anuales. El segundo pide crear un constructor para el concepto de fecha con sus atributos, día, mes y año, además de instanciar un objeto de esa clase para asignar y modificar la fecha. Asimismo, utilizar inteligencia artificial para consultar una decisión de diseño, usar o no setter y getter en el constructor.

1.2 Motivación

El control del acceso a las variables internas de los objetos. Aplicar técnicas de encapsulamiento y reglas de validación en la instanciación para evitar que el sistema opere con datos inconsistentes o erróneos, como remuneraciones financieras negativas. Por otro lado, analizar las respuestas generadas por modelos de lenguaje para diferenciar entre sugerencias de arquitectura correctas e imprecisiones conceptuales.

1.3 Objetivos

- Revisar y analizar las respuestas de distintas LLMs sobre el uso de modificadores de acceso y métodos modificadores en constructores.
- Implementar una clase Empleado en Java que aplique el encapsulamiento.
- Controlar la asignación de salarios mediante la validación de valores no negativos en la instanciación y modificación.
- Desarrollar una clase Fecha que organice datos temporales mediante constructores y métodos de acceso y modificación.

2. Marco Teórico

El paradigma de la Programación Orientada a Objetos (POO) busca modelar entidades del mundo real combinando estado (atributos) y comportamiento (métodos) dentro de clases. Para asegurar que los objetos mantengan un comportamiento predecible

durante todo su ciclo de vida, el paradigma se fundamenta en mecanismos de protección de datos [1].

El encapsulamiento es el principio mediante el cual se ocultan los detalles internos de implementación de una clase y se restringe el acceso directo a sus atributos desde el exterior. En Java, esto se logra utilizando modificadores de acceso, los cuales definen el nivel de visibilidad de los miembros de una clase: **private**, que oculta los atributos o métodos, permitiendo su acceso e invocación exclusivamente desde la misma clase. **public**, que expone los miembros a cualquier otra clase del proyecto, constituyendo la interfaz pública de comunicación del objeto. **package-private**, aplicado por defecto cuando no se especifica un modificador, limitando la visibilidad a las clases que pertenecen al mismo paquete. Y **protected**, que otorga acceso a las clases del mismo paquete y a las subclases derivadas mediante herencia. [2]

El constructor es un método especial de una clase cuya función principal es inicializar la memoria y establecer el estado inicial de un objeto al ser instanciado mediante el operador **new** [3]. Un objetivo crítico del constructor es asegurar la conservación de las invariantes de clase, las cuales son condiciones que deben cumplirse en todo momento para que un objeto se considere válido [1].

Dado que los atributos se declaran con visibilidad **private** para impedir su manipulación arbitraria, la lectura y actualización del estado de un objeto se realiza mediante métodos públicos conocidos como **getters** y **setters** [2]: Los Métodos de acceso (Getters) que devuelven el valor actual de un atributo privado sin modificar el estado interno. Y los Métodos modificadores (Setters) que reciben un parámetro para actualizar un atributo privado. Estos métodos permiten incorporar filtros o validaciones condicionales para descartar valores incompatibles con las reglas del programa antes de concretar la modificación [1].

3. Desarrollo

3.1 Actividad 1: Análisis de las respuestas de las LLMs

En esta actividad realizamos un análisis comparativo sobre las respuestas de las LLMs acerca del uso de modificadores de acceso y métodos modificadores (*setters*) dentro de constructores en Java.

Análisis de la Respuesta 1

En el primer prompt se consultó si es conveniente emplear métodos modificadores en constructores, añadiendo explícitamente entre paréntesis que se refería a los modificadores `public`, `private` y `protected`. Esta aclaración delimitó la respuesta hacia la visibilidad de los constructores debido a que presentó una confusión terminológica entre *métodos modificadores* (*setters*) y *modificadores de acceso*.

La LLM interpretó la especificación del paréntesis y enfocó su respuesta en el control de acceso para la instanciación de clases. Explicó detalladamente los niveles `public`, `private`, `protected` y por defecto, ejemplificando el uso de constructores privados junto con métodos de fábrica estáticos para la validación previa de objetos. Sin embargo, al ajustarse a la aclaración entre paréntesis, la LLM no abordó el problema del uso de métodos *setters* dentro de constructores, por lo que la cuestión sobre métodos modificadores no fue aclarada.

Análisis de la Respuesta 2

El segundo prompt cuestionó si es conveniente utilizar métodos modificadores (*setters*) dentro de los constructores y solicitó una justificación detallada de al menos media cuartilla sobre las implicaciones del problema. Este mismo empleó la terminología técnica adecuada y fijó una restricción de longitud.

La LLM determinó que el uso de métodos modificadores dentro de constructores es un **antipatrón** y una práctica no conveniente, especialmente si dichos métodos son públicos, protegidos o sobrescribibles (*virtuales*).

Justificó su postura explicando los riesgos del polimorfismo y el despacho dinámico en jerarquías de herencia: “Si una clase base llama a un *setter* sobrescribible en su constructor, se ejecutará el método de la subclase antes de que el constructor de dicha subclase inicialice sus propios atributos, generando errores como `NullPointerException`”. Además propuso el uso de métodos auxiliares `private` o `final`, validación directa con asignación de atributos y el patrón *Builder*.

Análisis de la Respuesta 3

El tercer prompt planteó una pregunta directa respecto a la conveniencia de usar métodos modificadores en constructores, exigiendo una justificación conceptual de al menos media cuartilla sin incluir especificaciones de formato.

La LLM desarrolló un texto fluido tipo ensayo descartando el uso de *setters* en constructores, explicando el problema del despacho dinámico en Java y C#, detallando

cómo la ejecución de métodos sobrescritos por una subclase causa estados inconsistentes. Abordó el riesgo de los efectos secundarios (registro de eventos, persistencia, invalidación de caché) cuando el objeto no se ha terminado de construir. Y mencionó problemas de acoplamiento de la API de modificación con el proceso de creación del objeto.

Al final recomendó una validación directa, métodos auxiliares privados o patrones *Builder* y fábricas estáticas.

Observaciones

El Prompt 1 presentó una ambigüedad terminológica al usar “métodos modificadores” para referirse a “modificadores de acceso” (`public`, `private`, `protected`). Los Prompts 2 y 3 incluyeron explícitamente una condición de longitud mínima (“media cuartilla”), mientras que el Prompt 1 solicitó diversos elementos (pros, contras, cambios sin modificador) con un lenguaje informal.

La Respuesta 1 tomó un camino totalmente distinto a las otras dos debido a la aclaración de visibilidad de constructores, enfocándose en la encapsulación de la instanciación. Comparando las soluciones que abordaron el mismo problema técnico, la Respuesta 2 y la Respuesta 3 coincidieron en que el uso de *setters* en constructores es una mala práctica (antipatrón). Ambas señalaron el despacho dinámico y los efectos secundarios como los principales riesgos.

3.2 Actividad 2: Programa Empleado

Inicialmente se implementó la clase Empleado, definiendo como atributos privados de tipo cadena el nombre y el apellido, así como una variable de tipo numérico flotante y doble para el salario mensual. A continuación, se construyó el método constructor recibiendo estos tres datos como parámetros; dentro del constructor, los datos se asignaron directamente mediante la referencia del objeto, mientras que el salario mensual se procesó a través de un método de validación interno que retorna un valor nulo de cero en caso de recibir una cantidad negativa.

Posteriormente, se definieron los métodos modificadores y de acceso para cada atributo. En el caso específico del método modificador del salario mensual, se incorporó una estructura condicional que verifica si el argumento recibido es estrictamente mayor a cero antes de actualizar la variable del objeto. De forma complementaria, se estructuraron dos métodos operativos: uno encargado de retornar el salario anual multiplicando el salario mensual por doce, y otro diseñado para actualizar la remuneración incrementando la variable del salario mensual en un diez por ciento mediante una multiplicación.

Finalmente, se desarrolló la clase principal de ejecución para realizar las pruebas

de funcionamiento. Inicialmente, el programa prepara la lectura desde la consola e ingresa los datos correspondientes al primer empleado, incluyendo la prueba con un saldo negativo para verificar el filtro de validación, seguido del ingreso de los datos del segundo empleado con valores válidos. Posteriormente, la aplicación invoca el método de cálculo anual para imprimir los ingresos de ambos empleados antes de la modificación. Por ultimo, se ejecutan los métodos de aumento salarial sobre ambos objetos y se realiza una segunda llamada al método de cálculo para proyectar las cifras anuales incrementadas en la salida estándar.

3.3 Actividad 3: Programa Fecha

Inicialmente, se diseñó la clase Fecha declarando como atributos privados de tipo entero las variables correspondientes al día, al mes y al año. A continuación, se redactó el método constructor de la clase, el cual recibe tres números enteros y los asigna directamente a sus respectivos campos bajo la premisa explícita de recibir valores previamente correctos.

Posteriormente, se agregaron los métodos de acceso (*getters*) para consultar individualmente cada componente numérico, así como los métodos modificadores (*setters*) para actualizar de manera independiente el día, el mes o el año. Adicionalmente, se implementó el método de despliegue, el cual concatena las variables enteras insertando barras diagonales como separadores para construir la representación del texto en el formato establecido de día, mes y año.

Finalmente, se implementó la clase de prueba para evaluar la manipulación del objeto instanciado. Para empezar, se solicita al usuario la captura consecutiva del día, mes y año a través del teclado y transfiere estos valores al constructor para crear la instancia inicial. Posteriormente, el programa invoca el método de despliegue para presentar la fecha registrada en la consola. A continuación, se solicitan nuevos datos numéricos al usuario y se envían de forma directa como argumentos a los métodos modificadores del objeto. Por último, la aplicación ejecuta nuevamente el método de despliegue para confirmar la actualización de la fecha modificada en la salida.

4. Resultados

4.1 Resultados de la Práctica 5

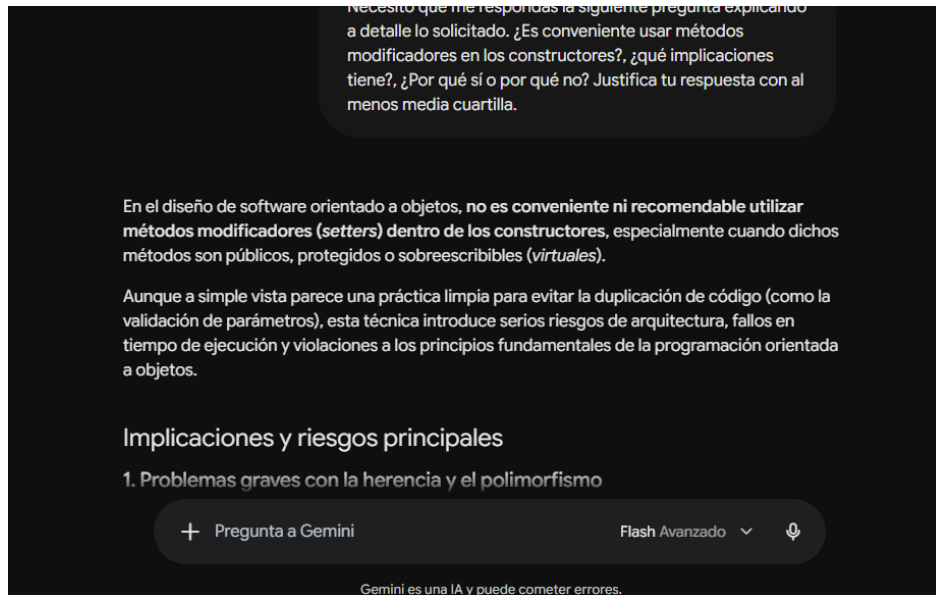
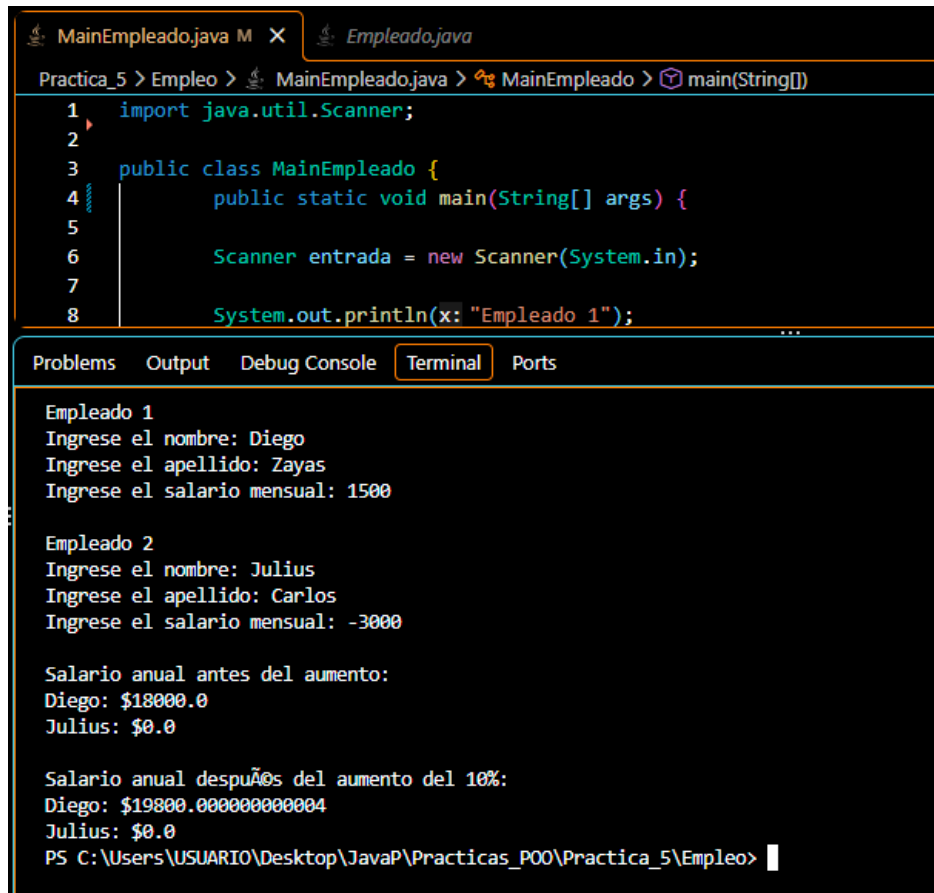


Figura 1: Captura de pantalla de la respuesta de una LLM sobre visibilidad y métodos modificadores en constructores.

Resultados de ejecución



The screenshot shows an IDE with two tabs: `MainEmpleado.java` and `Empleado.java`. The `MainEmpleado.java` tab is active, displaying the following code:

```
1 import java.util.Scanner;
2
3 public class MainEmpleado {
4     public static void main(String[] args) {
5
6         Scanner entrada = new Scanner(System.in);
7
8         System.out.println(x: "Empleado 1");
9     }
10 }
```

Below the code editor, the `Terminal` tab is selected, showing the execution output:

```
Empleado 1
Ingrese el nombre: Diego
Ingrese el apellido: Zayas
Ingrese el salario mensual: 1500

Empleado 2
Ingrese el nombre: Julius
Ingrese el apellido: Carlos
Ingrese el salario mensual: -3000

Salario anual antes del aumento:
Diego: $18000.0
Julius: $0.0

Salario anual después del aumento del 10%:
Diego: $19800.000000000004
Julius: $0.0
PS C:\Users\USUARIO\Desktop\JavaP\Practicas_POO\Practica_5\Empleo>
```

Figura 2: Captura de pantalla de la ejecución del programa Empleado en consola, mostrando la validación de salarios no negativos, el despliegue del salario anual inicial y el recálculo tras aplicar el incremento del 10 por ciento.

```
Practica_5 > Fecha > MainFecha.java > MainFecha > main(String[])
3 public class MainFecha {
5     public static void main(String[] args) {
24         fecha1.mostrarFecha();
25
26         // Modificar la fecha
27         System.out.println(x: "\nIngrese el nuevo día:");
28         fecha1.setDia(entrada.nextInt());
29
30         System.out.println(x: "Ingrese el nuevo mes:");
31         fecha1.setMes(entrada.nextInt());
32     }
33 }
```

Problems Output Debug Console **Terminal** Ports

```
Ingrese el día:
02
Ingrese el mes:
10
Ingrese el año:
2026
Fecha ingresada:
2/10/2026

Ingrese el nuevo día:
01
Ingrese el nuevo mes:
03
Ingrese el nuevo año:
2007

Fecha modificada:
1/3/2007
```

Figura 3: Captura de pantalla de la ejecución del programa Fecha en consola, evidenciando la lectura de valores, la salida en formato día/mes/año y la posterior actualización de la instancia mediante métodos modificadores.

Diagrama 01: Diagrama de Clases UML

[Ver imagen 01_Diagrama_Clases_UML.png en GitHub](#)

Diagrama 02: Fecha mostrarFecha

[Ver imagen 02_Fecha_mostrarFecha.png en GitHub](#)

Diagrama 03: Empleado validarSalario

[Ver imagen 03_Empleado_validarSalario.png en GitHub](#)

Diagrama 04: MainFecha main

[Ver imagen 04_MainFecha_main.png en GitHub](#)

Diagrama 05: MainEmpleado main

[Ver imagen 05_MainEmpleado_main.png en GitHub](#)

5. Conclusiones

Los resultados obtenidos en la ejecución de los programas demuestran que la aplicación del encapsulamiento y la validación de entradas garantizan la integridad del estado de los objetos. En el caso del programa Empleado, la evaluación condicional de los salarios dentro del constructor y de los métodos modificadores impidió la existencia de montos financieros negativos, asegurando que las operaciones posteriores, como el cálculo del salario anual y el incremento del diez por ciento, partieran de cifras numéricamente válidas.

Por su parte, el diseño utilizado en la clase Fecha confirmó que separar la representación interna de los datos respecto a su despliegue facilita la actualización controlada de sus atributos sin alterar la estructura del programa. La interacción observada en ambas actividades evidencia que los métodos de acceso y modificación, junto con los modificadores de acceso privados, son indispensables para proteger las invariantes de clase y evitar inconsistencias en el flujo de datos.

6. Referencias

- [1] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ, USA: Prentice Hall, 2008. [En línea]. Disponible en:
<https://www.lkhibra.ma/books/clean-code.pdf>
- [2] Oracle, “Controlling Access to Members of a Class,” *The Java™ Tutorials*, 2022. [En línea]. Disponible en:
<https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html>
- [3] Oracle, “Providing Constructors for Your Classes,” *The Java™ Tutorials*, 2022. [En línea]. Disponible en:
<https://docs.oracle.com/javase/tutorial/java/javaOO/constructors.html>